

NASA C-MAPSS FD001 Predictive Maintenance

ADIF71-2627PSA01 - Machine Learning I

Prepared by :

HIRCHYTS Daniil
LOUET Matteo
SINGERY Stéphane

EFREI Paris

ING2 APP : BDML 2

Academic Year 2026 - 2027

14 October 2026

Table des matières

Executive summary **2**

I. Project objective and dataset scope **2**
1.1 Dataset variables and sensor meanings . . . 3
1.2 Definition of the endpoint test . . . 4

II. Data loading and quality assurance **4**
2.1 Quality checks . . . 4
2.2 One engine and one recorded life . . . 5

III. Engine lifetimes and validation design **6**
3.1 Why validation must group complete engines . . . 7

IV. Target construction and warning window **7**
4.1 Training labels . . . 7
4.2 Test labels and alignment . . . 8
4.3 Sensitivity to the window . . . 9

V. Screening the operational settings and sensors **9**

VI. Degradation patterns and class separation **11**
6.1 Sensor trajectories as failure approaches . . . 11
6.2 Standardisation and distributions by class . . . 12

VII. Outlier analysis **15**

VIII. Correlation analysis and feature selection **17**
8.1 Ranking by remaining-life association . . . 18

IX. Modelling protocol and baseline **20**
9.1 Grouped validation and prediction scores . . . 20
9.2 Always healthy reference . . . 20

X. Comparison of the classifier families **21**

XI. Sensor ablation and class weighting **23**
11.1 Testing whether s9 and s14 can be removed . . . 23
11.2 Effect of balanced class weights . . . 24

XII. Hyperparameter tuning **24**
12.1 Reevaluation on changed engine shuffles . . . 25

XIII. Model selection and alarm threshold **26**
13.1 Choosing the reference estimator . . . 26
13.2 Selecting an operating point from training predictions . . . 27

XIV. Final evaluation and uncertainty **28**
14.1 Refitting and testing on unseen engines . . . 28
14.2 Bootstrap uncertainty . . . 29
14.3 All rows compared with endpoints . . . 30

XV. Error analysis and continuous monitoring **30**
15.1 The seven endpoint errors . . . 30
15.2 Alarm frequency along the test histories . . . 31

XVI. Conclusions limitations **32**
16.1 Limitations of the current study . . . 32

Executive summary

We study whether a simulated turbofan engine will reach failure within 30 operating cycles using the sensor readings available at its current cycle. The report documents the complete FD001 workflow, from parsing and validating the raw files to exploratory analysis, feature selection, seven classifier families, hyperparameter search, alarm threshold selection and evaluation on previously unseen test engines.

The selected model is a logistic regression with standardisation, balanced class weights and C = 100. At an alarm threshold of approximately 0.592, it detects 21 of the 25 test engines with an imminent failure, misses four, and raises three false alarms among 75 engines outside the warning window. Recall is 0.840, precision 0.875 and F1 0.857. The ROC AUC is 0.978.

Measure	**Result**
Training engines and candidate sensors	100 engines and 14 sensors
Test population	100 engines with 25 imminent failures
True positives and false negatives	21 and 4
False positives and true negatives	3 and 72
Recall and precision	0.840 and 0.875
F1 and ROC AUC	0.857 and 0.978

Table 1 – Selected alarm performance on the last recorded row of each test engine

The sensor analysis supports retaining all 14 informative channels, including s9 and s14. Removing these two weaker individual predictors lowers average precision for every tested model. The main limitation is uncertainty : only 25 test engines are positive, and the bootstrap recall interval is 0.667 to 0.963. Row-level monitoring also produces lower precision than the endpoint test. These results support a useful FD001 reference model, but do not establish a validated maintenance policy for operational aircraft.

I. Project objective and dataset scope

The project reformulates remaining useful life estimation as binary classification. For each engine-cycle record, the output is an imminent-failure label : $y = 1$ when the remaining useful life is at most 30 cycles, and $y = 0$ otherwise. The model sees a single sensor snapshot. It does not use a sequence of previous flights, future measurements or the true remaining life as an input.

We also developed an interactive website to help readers better understand the predictive maintenance problem and the engine degradation studied in this project. This companion resource is available at https://nasa-c-mapss.hirchyts-daniil.me/.

FD001 is a simulated subset of NASA C-MAPSS with one operating condition and one fault mode, high-pressure compressor degradation. Each training trajectory continues until failure, whereas each test trajectory stops before failure and has a supplied remaining-life value for its endpoint [1, 2]. An engine cycle is treated as the discrete operational unit in this dataset ; the analysis does not convert a cycle into a duration in hours.

File	Records	Role
train_FD001.txt	20,631 rows 100 engines	Complete trajectories used for exploration and model fitting
test_FD001.txt	13,096 rows 100 engines	Truncated trajectories reserved for final model evaluation
RUL_FD001.txt	100 values	Remaining life after the last recorded cycle of each test engine

Table 2 – FD001 files and their roles

Every input row has 26 columns : unit identifier, cycle number, three operational settings and 21 sensor readings. The training and test unit identifiers both run from 1 to 100, but identify different physical engines. Consequently, a numerical identifier shared between files does not imply a shared trajectory. The supplied readme contains a numbering inconsistency in its final sensor example ; the actual schema has 21 sensor channels.

Column position	Notebook name	Content
1	`unit`	Engine identifier
2	`cycle`	Recorded operational cycle
3	`setting_1`	Operational setting 1
4	`setting_2`	Operational setting 2
5	`setting_3`	Operational setting 3
6–26	`s1`–`s21`	Sensor measurements 1–21, detailed below

Table 3 – Ordered schema of the 26 FD001 raw columns

1.1 Dataset variables and sensor meanings

`unit` identifies an engine and `cycle` identifies each successive recorded flight. The three operational settings (`setting_1` to `setting_3`) describe the simulated operating condition ; their physical units are not specified in the dataset. The 21 sensor channels are numbered `s1` to `s21`. The sensor names and units below follow the mapping used in notebook section 1.2. The raw FD001 files number the sensors but do not provide these names ; the notebook maps them by the order of the sensor descriptions in Saxena et al. (2008, Table 2). LPC and HPC mean low- and high-pressure compressor ; HPT and LPT mean high- and low-pressure turbine.

Column	Symbol	Physical meaning	Unit
s1	T2	Total temperature at fan inlet	°R
s2	T24	Total temperature at LPC outlet	°R
s3	T30	Total temperature at HPC outlet	°R
s4	T50	Total temperature at LPT outlet	°R
s5	P2	Pressure at fan inlet	psia
s6	P15	Total pressure in bypass-duct	psia
s7	P30	Total pressure at HPC outlet	psia
s8	Nf	Physical fan speed	rpm
s9	Nc	Physical core speed	rpm
s10	epr	Engine pressure ratio (P50/P2)	–
s11	Ps30	Static pressure at HPC outlet	psia
s12	phi	Ratio of fuel flow to Ps30	pps/psi
s13	NRf	Corrected fan speed	rpm
s14	NRc	Corrected core speed	rpm
s15	BPR	Bypass ratio	–
s16	farB	Burner fuel-air ratio	–
s17	htBleed	Bleed enthalpy	–
s18	Nf_dmd	Demanded fan speed	rpm
s19	PCNfR_dmd	Demanded corrected fan speed	rpm
s20	W31	HPT coolant bleed	lbm/s
s21	W32	LPT coolant bleed	lbm/s

TABLE 4 – FD001 sensor channels and their physical interpretation

The subsequent screening retains 14 sensor channels as model inputs : s2, s3, s4, s7, s8, s9, s11, s12, s13, s14, s15, s17, s20, s21. The exclusion of the other channels and the operational settings is justified in the feature-screening section.

1.2 Definition of the endpoint test

The primary test contains one prediction per test engine at its last recorded cycle. This matches the point at which the answer key supplies remaining life. A second analysis scores all test rows to approximate the challenges of continuous monitoring. The two views have different class prevalence and must not be treated as interchangeable estimates of precision.

II. Data loading and quality assurance

We parse the whitespace-separated text files without a header and assign an explicit ordered schema. The RUL file is matched to test engines by its line number. Its unit index is created as row index plus one, then joined to the last recorded cycle for each engine. The merge is checked as one-to-one, the set of engine identifiers must match, and all answer values must be present and non-negative.

2.1 Quality checks

The quality checklist tests both trajectory files for missing values, duplicated complete rows, non-numeric columns, cycles that do not start at one, and gaps in each engine sequence. All

ten checks pass. No imputation or categorical encoding is needed. These checks establish the structural validity of the supplied files ; they do not certify that simulated signals reproduce every feature of real engine operation.

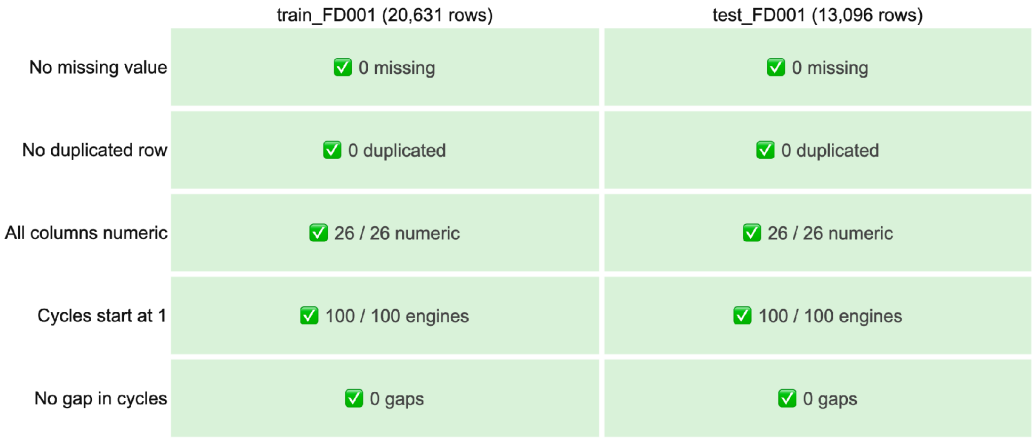


FIGURE 1 – Data quality checklist for the training and test trajectory files
Source : notebook section 2.1.

Descriptive statistics are inspected for all three settings and all 21 sensors. Their means, standard deviations and ranges reveal very different measurement scales. For example, s11 is near 48, whereas rotational speed channels can be near 9,000. This motivates standardisation for the linear, distance and kernel models rather than direct comparison of raw amplitudes.

2.2 One engine and one recorded life

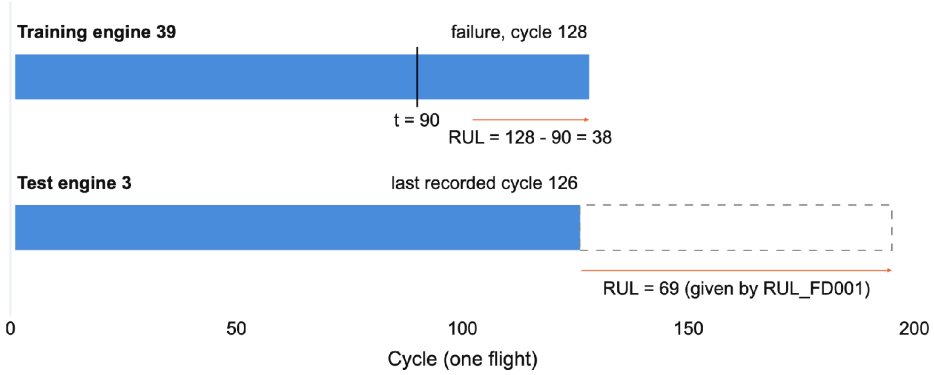


FIGURE 2 – Complete training life and truncated test life shown on a cycle axis
Source : notebook section 3.2.

The example separates recording length from lifetime. Engine 39 in training fails at cycle 128 ; at cycle 90 it has 38 cycles left. Test engine 3 ends at cycle 126 and has 69 cycles remaining according to the answer key, so its reconstructed failure cycle is 195. Treating the last test cycle as failure would create incorrect labels and invalid model evaluation.

III. Engine lifetimes and validation design

Training lifetimes range from 128 to 362 cycles, with a mean of 206.3, a median of 199 and a standard deviation of 46.3. The longest life is about 2.8 times the shortest. A cycle count alone therefore cannot determine how close a particular engine is to failure ; sensor condition is required to distinguish engines at the same age.

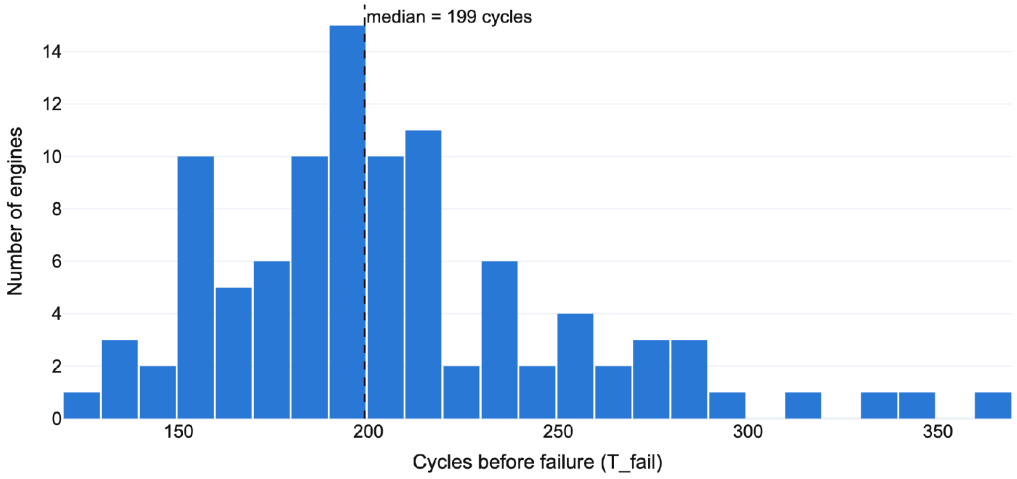


FIGURE 3 – Distribution of complete lifetimes across the 100 training engines
Source : notebook section 4.1.

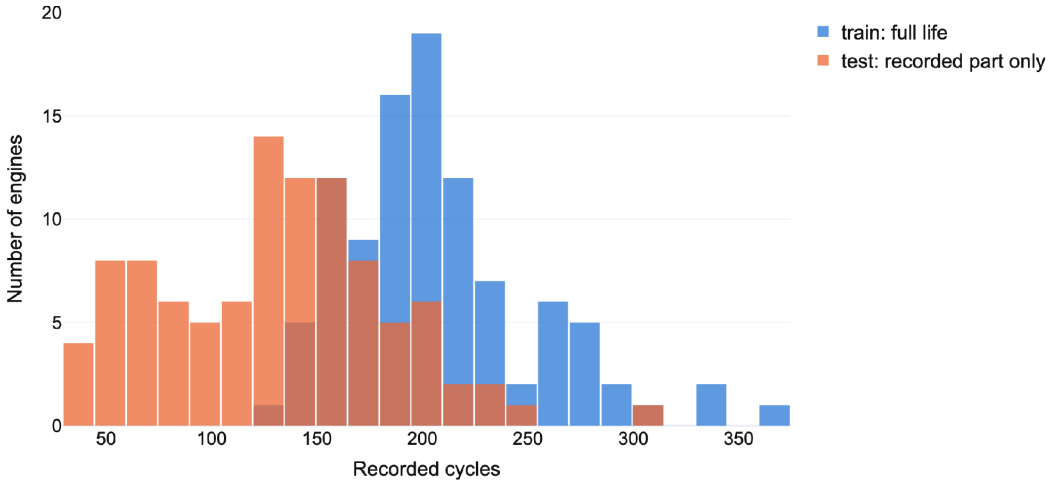


FIGURE 4 – Recorded trajectory lengths in training and test with different recording endpoints
Source : notebook section 4.2.

Test recordings range from 31 to 303 cycles and have a median length of 134. Their endpoint RUL ranges from seven to 145 cycles, with median 86. After adding the answer-key RUL, the reconstructed median test lifetime is 199 cycles, matching training. No exact row of settings and sensor values is shared across the two files. These observations are consistent with separate engines from a similar simulation regime, but do not prove that every aspect of their distributions is identical.

3.1 Why validation must group complete engines

Measurements within an engine are highly dependent. A random row split can place adjacent cycles of the same engine on opposite sides of the validation boundary. In the illustrated 20 percent row split, all 100 engines appear on both sides and 97 percent of validation rows have a neighbouring cycle in training. That split would measure interpolation within known trajectories rather than performance on a new engine.

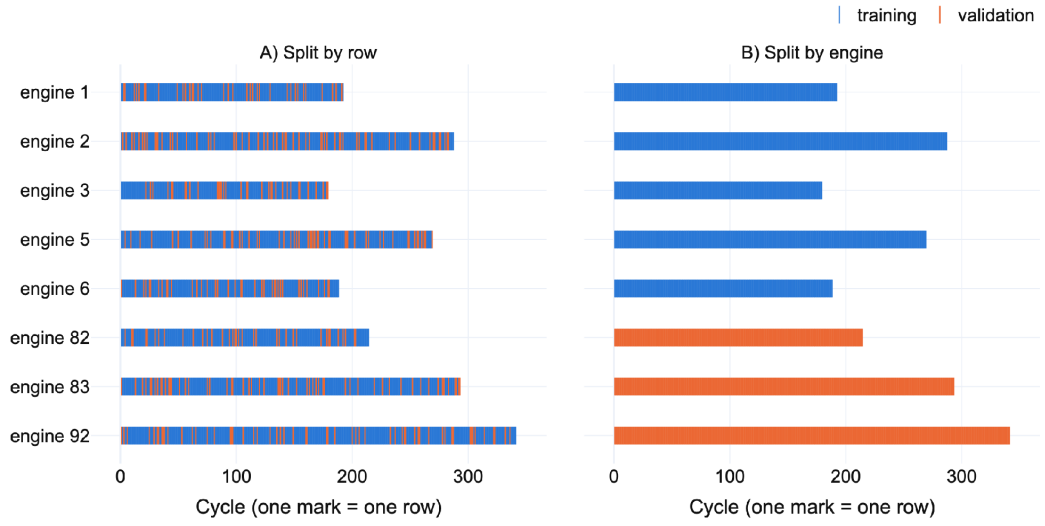


FIGURE 5 – Comparison of random row splitting and complete-engine splitting
Source : notebook section 4.3.

The engine split assigns all rows of an engine to one side : the example holds out 20 engines and produces zero shared engines and zero neighbouring-cycle contamination. Subsequent experiments use GroupKFold with unit as the grouping variable. Under five folds, each validation fold contains 20 complete engines. Scaling and fitting occur inside the training side of each fold.

IV. Target construction and warning window

4.1 Training labels

A training engine is observed until failure, so its final cycle is the known failure time. We compute a remaining-life value for every row by subtracting the current cycle from the final cycle for that engine, then apply the warning window.

$$\mathrm{RUL}_i(t) = T_{\mathrm{fail},i} - t$$

$$y = I(\mathrm{RUL} \leq w) \qquad w = 30$$

The indicator I is one when its condition holds and zero otherwise. The inclusive boundary matters : a complete trajectory contains 31 positive rows with RUL values from zero through 30. Across the 100 training engines this gives 3,100 positive rows and 17,531 negative rows, or 15.0 percent positives.

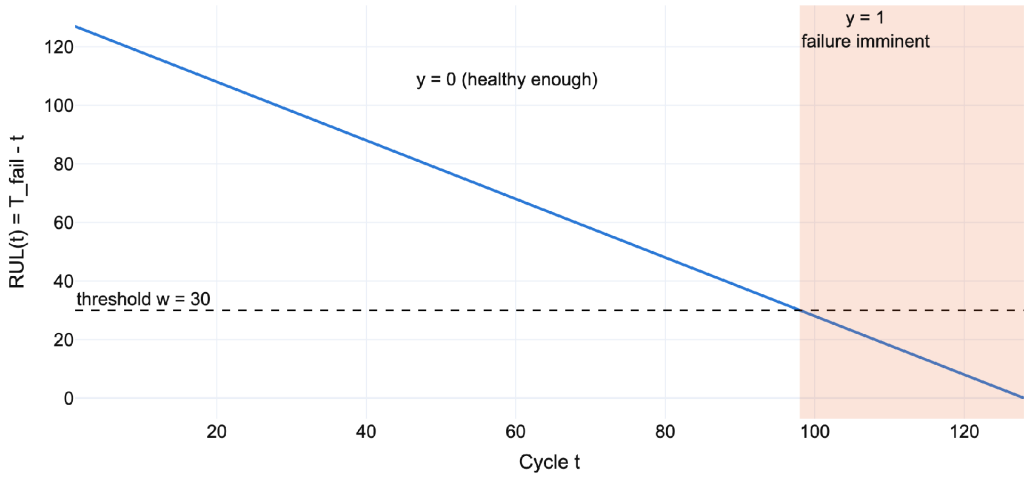


FIGURE 6 – Remaining life and the positive-label interval for one complete training trajectory
Source : notebook section 5.4.

4.2 Test labels and alignment

For test engine i, the endpoint answer provides RULfile,i after its last recorded cycle. At an earlier recorded cycle t, the remaining life includes the cycles still recorded before that endpoint. We use the following reconstruction for scoring the full test history.

$$\mathrm{RUL}_{\mathrm{test},i}(t) = (t_{\mathrm{last},i} - t) + \mathrm{RUL}_{\mathrm{file},i}$$

This reconstruction produces 332 positive test rows out of 13,096, or 2.535 percent. At the last recorded row, 25 of 100 engines are positive. The endpoint task has much higher prevalence than full-history monitoring. The RUL values are used only to create or evaluate the targets ; they are excluded from model features.

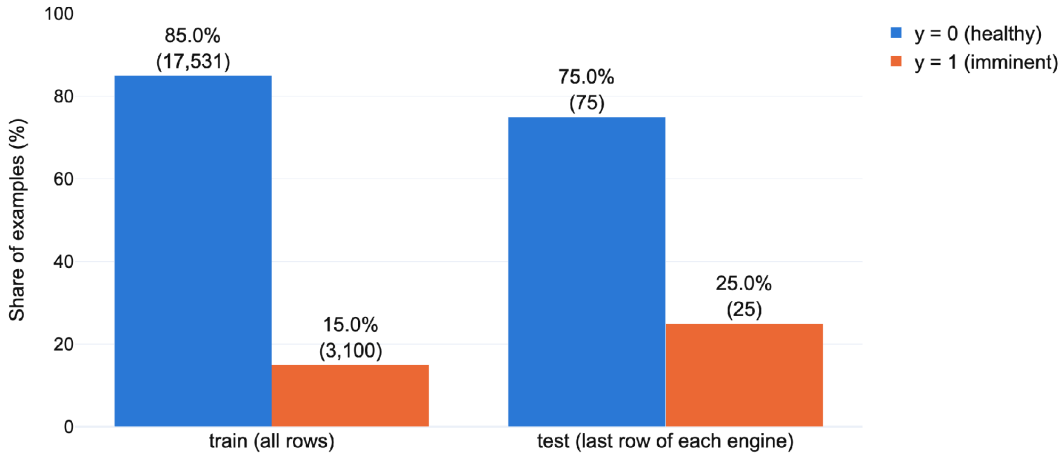


FIGURE 7 – Class proportions in all training rows and at the test endpoints
Source : notebook section 5.5.

4.3 Sensitivity to the window

Window in cycles	Training rows positive percent	Test endpoints positive percent
10	5.3	7.0
20	10.2	16.0
30	15.0	25.0
40	19.9	28.0
50	24.7	33.0

TABLE 5 – Positive-class proportions for alternative warning windows

The 30-cycle window is a modelling choice rather than a physical discontinuity. Increasing the window includes less degraded observations in the positive class and changes both prevalence and class separation. The s11 example shows a mean separation of about 2.50 within-class standard deviations at 30 cycles, falling to 1.85 at 75 cycles. These sensitivity calculations do not constitute a complete comparison of retrained models across windows.

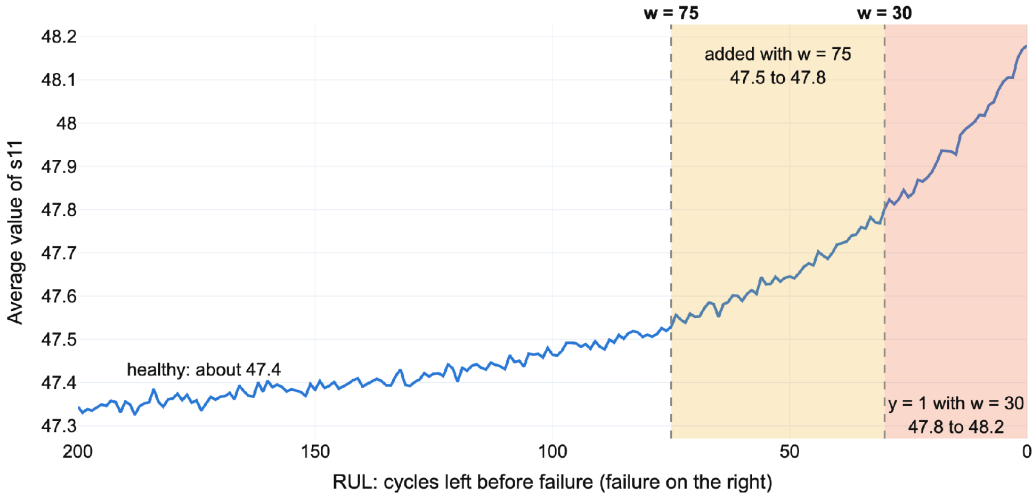


FIGURE 8 – How a wider window adds progressively less degraded observations to the positive class
Source : notebook section 5.8.

V. Screening the operational settings and sensors

We first screen columns for almost constant behaviour. A column is dropped when its most frequent value occurs in at least 95 percent of training rows. This avoids using a floating-point standard deviation alone to decide whether a channel carries useful variation. The calculation uses the training file ; it does not select features from test outcomes.

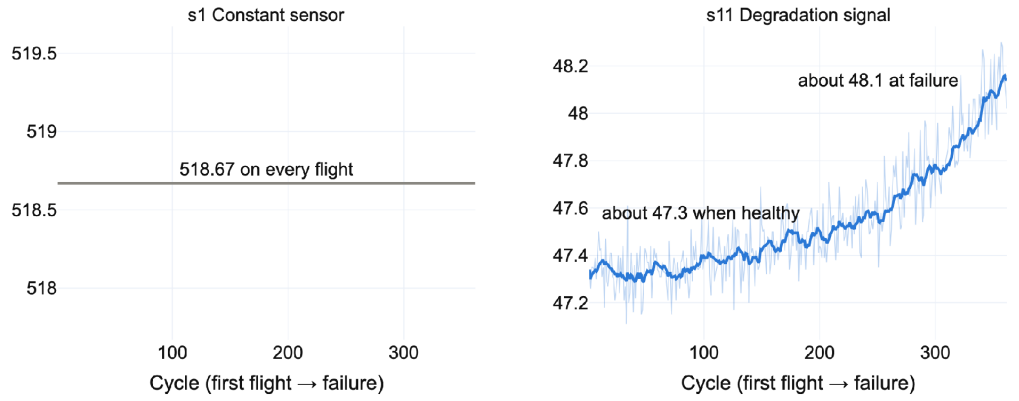


FIGURE 9 – A constant sensor compared with a sensor that changes during engine degradation
Source : notebook section 6.1.

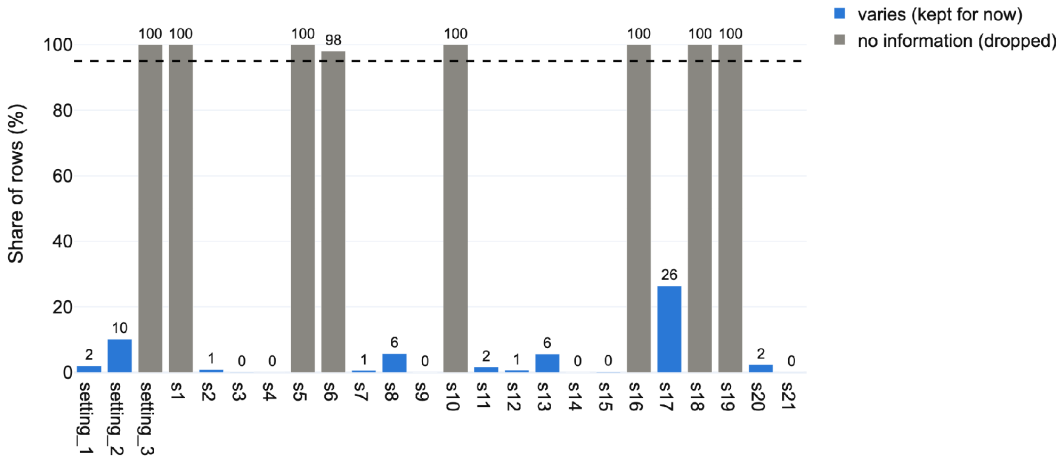


FIGURE 10 – Modal-value proportion for every operational setting and sensor with the 95 percent screening boundary

Source : notebook section 6.2.

Group	Columns	Reason
Almost constant	setting_3 ; s1 ; s5 ; s6 ; s10 ; s16 ; s18 ; s19	One value dominates the training measurements
Varying settings	setting_1 ; setting_2	Variation is very small and correlation with RUL is approximately zero
Identifiers and targets	unit ; RUL ; y	Identifiers are not condition measurements and targets would leak the answer
Age	cycle	Excluded from the starting feature set ; age alone is insufficient
Retained sensors	s2 ; s3 ; s4 ; s7 ; s8 ; s9 ; s11 ; s12 ; s13 ; s14 ; s15 ; s17 ; s20 ; s21	Fourteen informative candidates passed to modelling

Table 6 – Excluded and retained inputs

The standard deviations of setting_1 and setting_2 are about 0.002187 and 0.000293, and their correlations with RUL are -0.0032 and -0.0019. They are excluded for this single-condition FD001 problem. This decision should be revisited for subsets with multiple operating regimes, where settings can be needed to distinguish regimes or normalise measurements.

VI. Degradation patterns and class separation

6.1 Sensor trajectories as failure approaches

We align training observations by true RUL and inspect the last 200 cycles before failure. Fleet means are plotted alongside three representative engines with short, typical and long lifetimes. Several channels stay comparatively flat early and then drift more rapidly near failure. Other channels decrease. This shape explains why a simple relationship with calendar age is inadequate and why nonlinear model families are worth comparing.

The figure must be read with failure on the right, because the horizontal RUL axis is reversed. The main increasing family contains s2, s3, s4, s8, s11, s13, s15 and s17 ; s7, s12, s20 and s21 form a decreasing family. Their vertical scales differ, so the amplitude of a raw channel cannot be compared directly with another channel. The grey curves also show that an engine can be noisier or offset from the fleet mean. Aligning these complete training trajectories by their known failure time helps reveal the patterns ; it does not give the prediction model access to a future failure time.

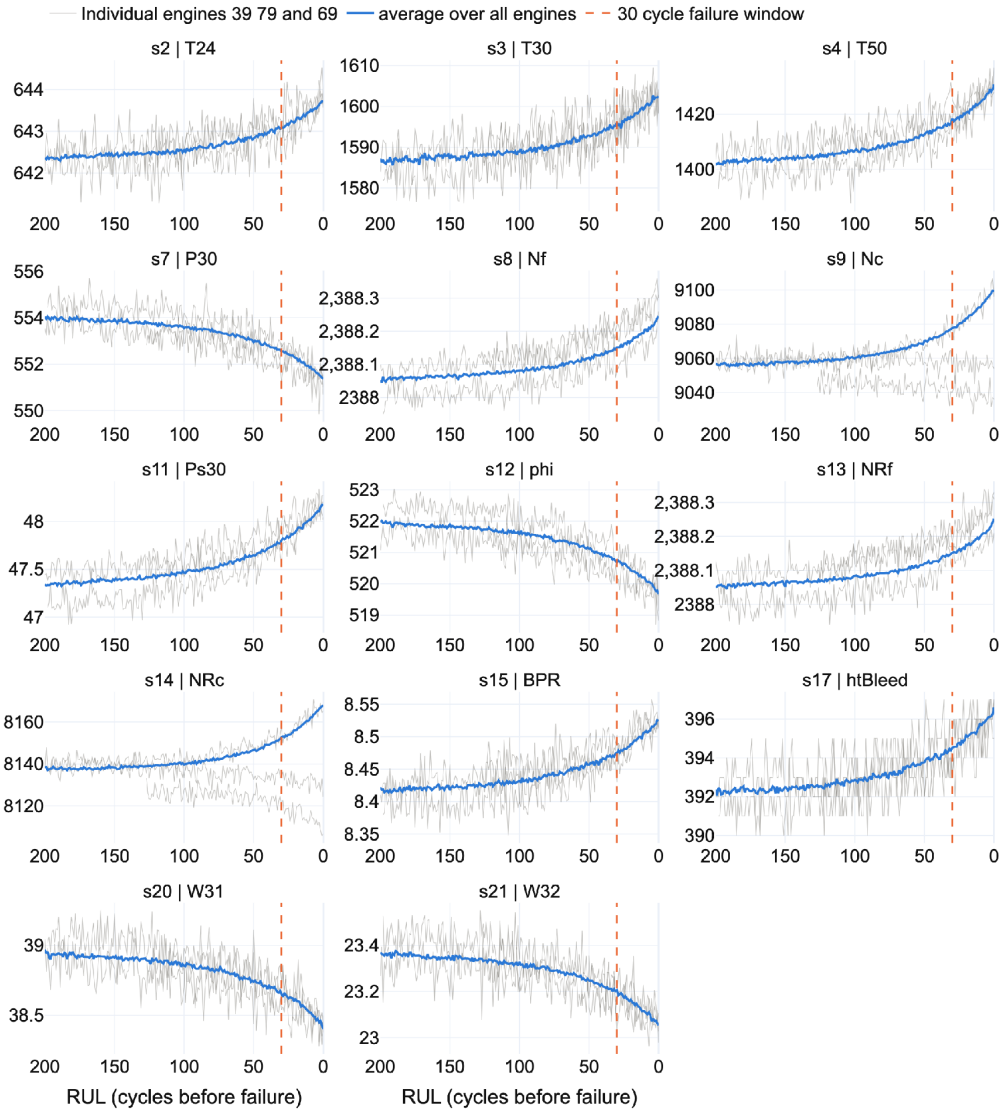


FIGURE 11 – Fourteen retained sensor trajectories aligned by remaining life with representative engines and the 30-cycle boundary

Source : notebook section 7.1.

The fleet average smooths the noise and is an exploratory description, not an online feature. Individual engines can depart substantially from it. In particular, s9 and s14 have marked differences between trajectories. Their later usefulness must be assessed in a multivariate classifier rather than decided solely from an average curve.

6.2 Standardisation and distributions by class

$$z = \frac{x - \mu}{\sigma}$$

For exploratory plots we express sensor readings as z-scores so their class distributions can be compared on a shared scale. The displayed exploratory transformation is fitted to the full training file. Modelling uses separate StandardScaler steps fitted within each training fold, preventing

validation observations from influencing the fitted means and standard deviations.

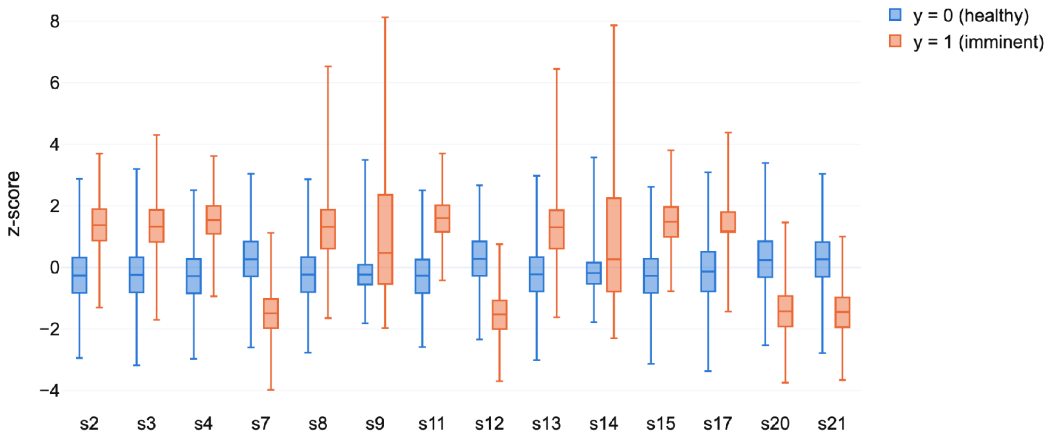


FIGURE 12 – Standardised sensor distributions for healthy and imminent-failure training rows
Source : notebook section 8.1.

The difference between class means is largest for s11 at +1.86 standard deviations and s4 at +1.81, followed in magnitude by s12 at −1.79 and s7 at −1.75. Signs indicate the direction of the condition change. The distributions overlap, so a single measurement does not perfectly separate the classes.

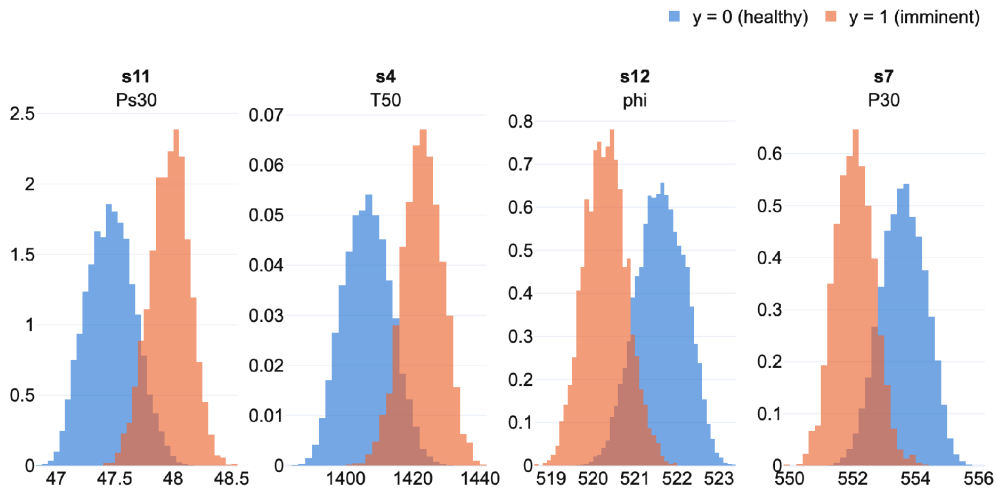


FIGURE 13 – Class histograms for the four sensors with the largest standardised mean differences
Source : notebook section 8.2.

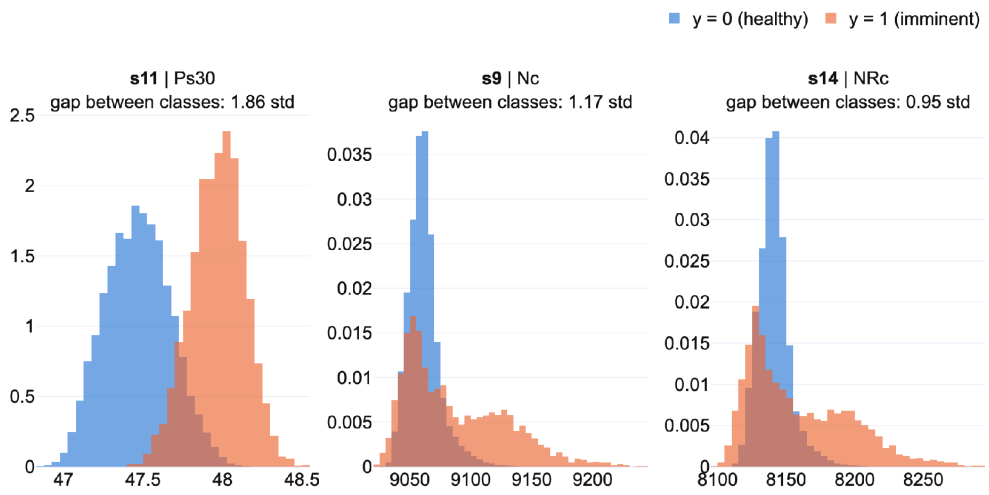


FIGURE 14 – Comparison of s11 with the weaker individual separators s9 and s14
Source : notebook section 8.3.

The imminent-failure spread of s9 is about 3.1 times its healthy-class spread, and the corresponding ratio for s14 is about 3.2. The ratio is only 0.8 for s11. A difference in spread is information that a ranking based only on mean separation can miss. This is one reason to retain s9 and s14 until an explicit ablation experiment has been performed.

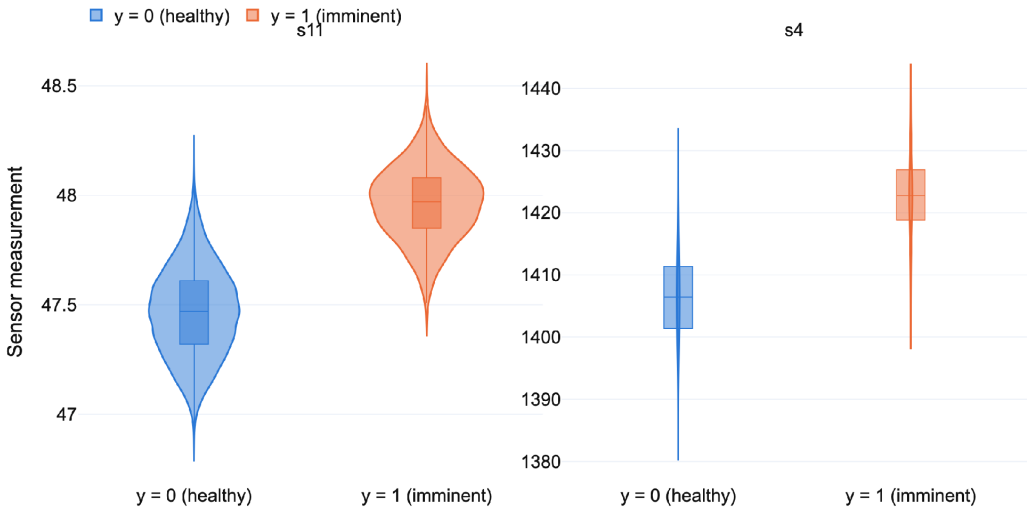


FIGURE 15 – Violin distributions in the two selected original sensor channels
Source : notebook section 8.4.

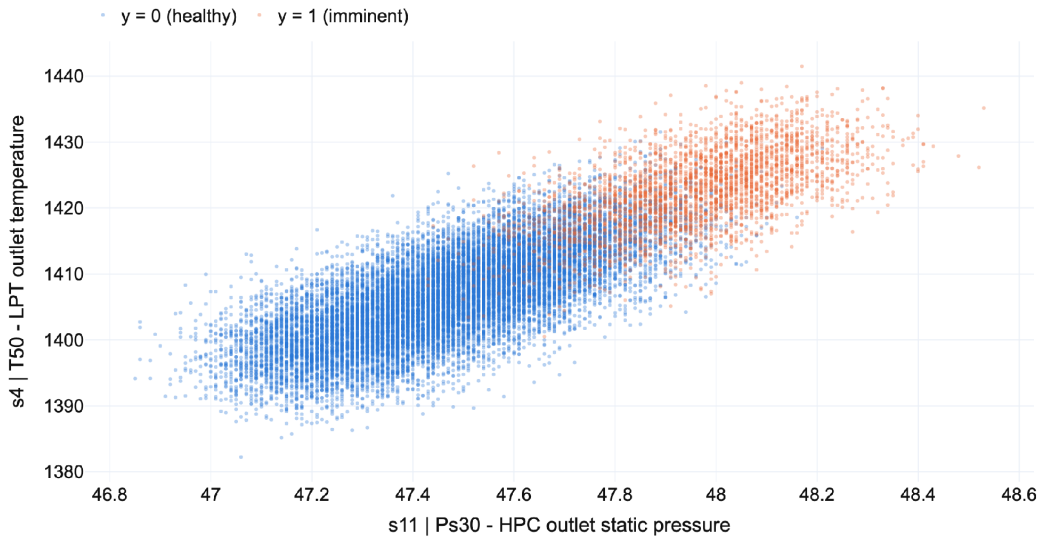


FIGURE 16 – Two-dimensional class separation in the original sensor space
Source : notebook section 8.4.

The two-dimensional view selects informative sensors while avoiding a highly correlated pair when possible. It confirms that the positive class tends to occupy a different region, but considerable overlap remains. This is a descriptive training view : it does not establish the test performance of any particular decision boundary.

VII. Outlier analysis

We use the interquartile-range rule to identify extreme readings : IQR = Q3 − Q1 ; a value is flagged when it is below Q1 − 1.5 IQR or above Q3 + 1.5 IQR. The generic boxplot below explains this rule using illustrative values rather than engine measurements. The next chart applies it to the actual s11 distribution.

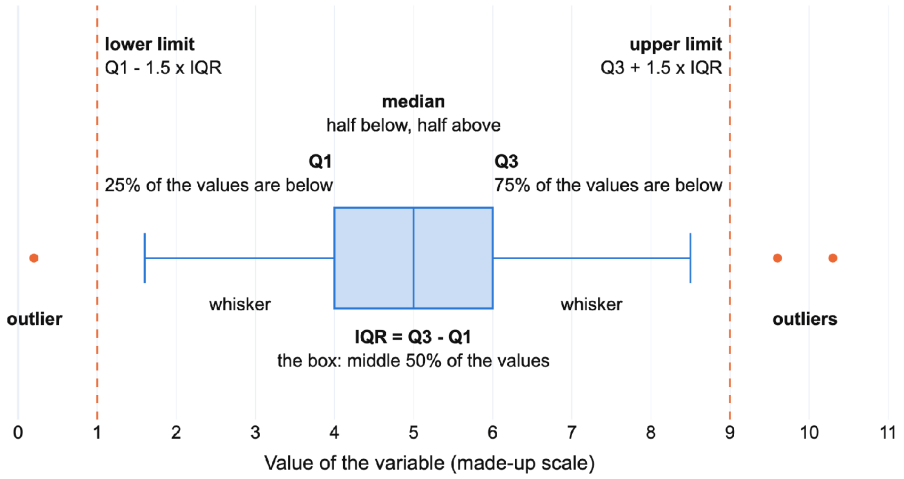


FIGURE 17 – Illustrative boxplot showing quartiles whiskers and outlier boundaries on a made-up scale
Source : notebook section 9.2.

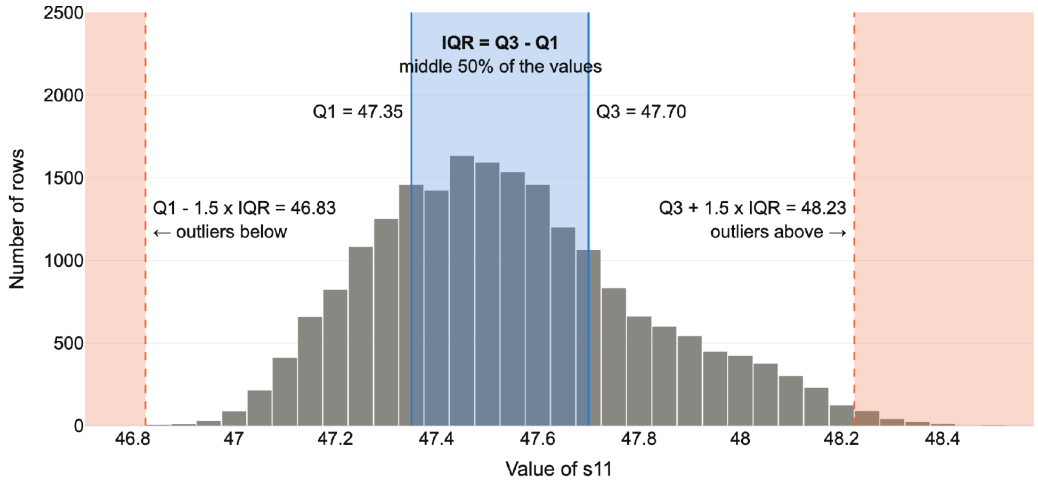


FIGURE 18 – Interquartile-range boundaries applied to the observed s11 measurements
Source : notebook section 9.3.

At least one retained sensor flags 2,431 training rows, or 11.8 percent of the file. Many flags occur near failure rather than being random isolated anomalies. For s11, s12 and several other channels, almost all flagged observations belong to the imminent-failure class. Removing them would discard part of the degradation signal and disproportionately remove positive examples.

Sensor	**Flagged rows**	**Rows percent**	**Flags within 30 cycles percent**	**Median RUL**
s9	1686	8.17	71.2	20.0
s14	1543	7.48	73.3	19.0
s8	320	1.55	97.2	5.0
s11	167	0.81	100.0	2.0
s3	165	0.8	86.7	6.0
s13	161	0.78	97.5	4.0
s12	146	0.71	100.0	3.0
s21	136	0.66	92.6	4.0
s2	128	0.62	89.1	6.0
s4	120	0.58	99.2	3.0
s15	120	0.58	94.2	4.5
s20	117	0.57	97.4	3.0
s7	110	0.53	98.2	2.0
s17	81	0.39	97.5	3.0

TABLE 7 – Outlier counts and proximity to failure for the retained sensors

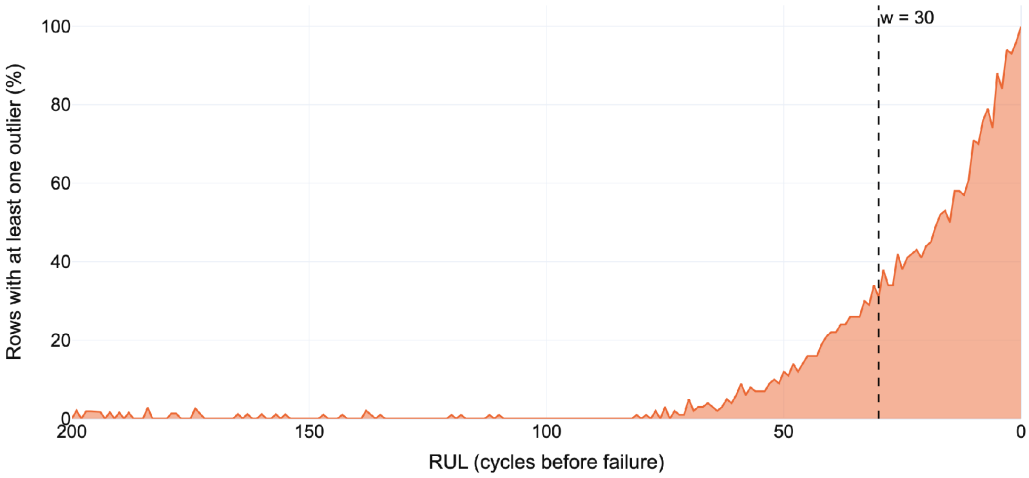


FIGURE 19 – Frequency of IQR outlier flags as a function of remaining life
Source : notebook section 9.5.

We retain these observations in the reference study. An outlier flag is not sufficient evidence of a corrupt measurement. This decision does not imply that sensor failures or physically impossible values should always be retained in a real system ; a deployment would need separate plausibility checks and a fault-handling policy.

VIII. Correlation analysis and feature selection

The Pearson matrix reveals groups of channels that move together as failure approaches. Twelve sensors have substantial mutual correlations, often with absolute magnitudes around 0.6 to 0.85. The s9 and s14 pair is strongly correlated with itself, approximately 0.96, but comparatively weakly correlated with the other channels. Correlation is an association ; it does not establish the physical cause of a measured change.

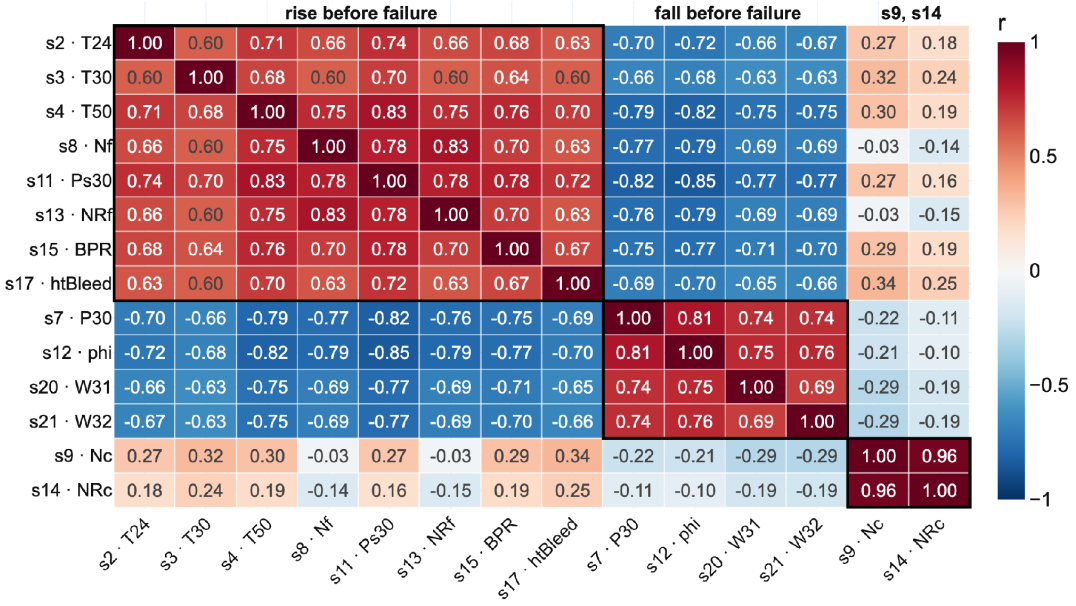


FIGURE 20 – Pearson correlation matrix ordered into increasing decreasing and s9 s14 sensor families
Source : notebook section 10.1.

The strong correlations between selected sensors raise concerns for both logistic regression and Gaussian naïve Bayes. They can make logistic-regression coefficients unstable ; if the correlations remain within each class, they also conflict with naïve Bayes's independence assumption. Nevertheless, correlation alone is not enough to discard either model. Both are simple models, so we will test their performance empirically with grouped cross-validation.

8.1 Ranking by remaining-life association

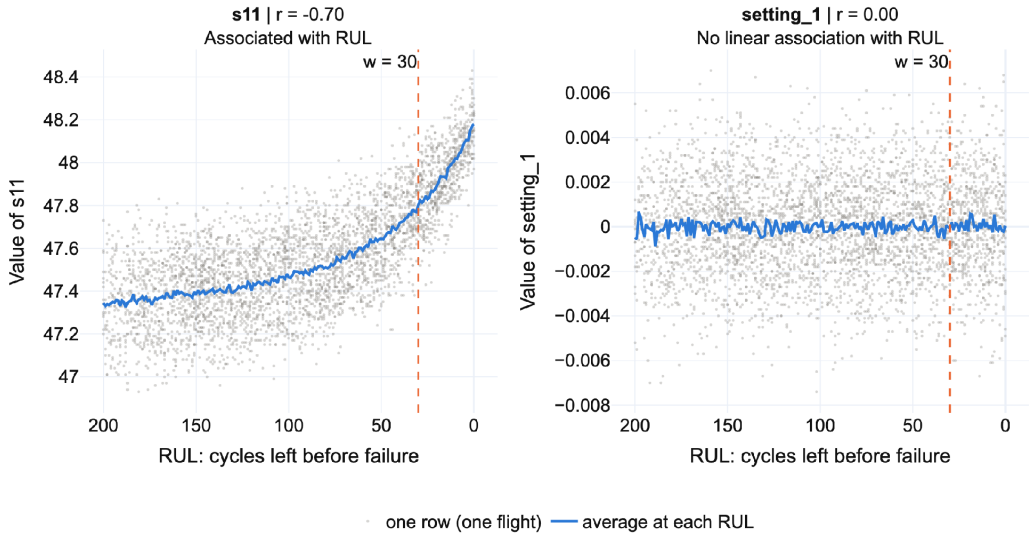


FIGURE 21 – A sensor associated with remaining life compared with an operational setting with no such association

Source : notebook section 11.1.

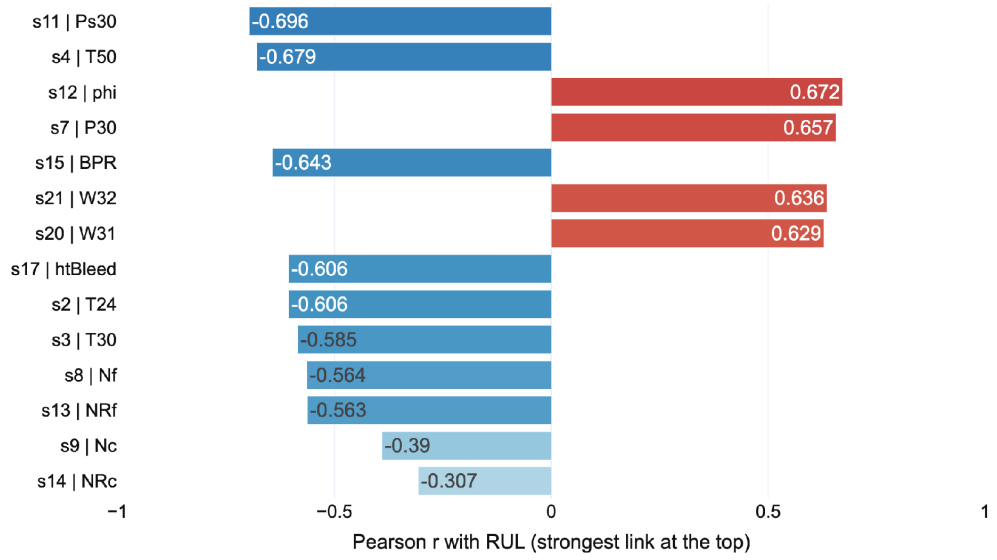


FIGURE 22 – Ranking of retained sensors by Pearson correlation with remaining life
Source : notebook section 11.2.

Sensor	Correlation with RUL	Absolute correlation with RUL	Correlation with y
s11	-0.696	0.696	0.666
s4	-0.679	0.679	0.648
s12	0.672	0.672	-0.640
s7	0.657	0.657	-0.626
s15	-0.643	0.643	0.619
s21	0.636	0.636	-0.606
s20	0.629	0.629	-0.600
s17	-0.606	0.606	0.583
s2	-0.606	0.606	0.581
s3	-0.585	0.585	0.562
s8	-0.564	0.564	0.543
s13	-0.563	0.563	0.540
s9	-0.390	0.390	0.420
s14	-0.307	0.307	0.341

TABLE 8 – Training correlations with remaining life and the binary target

The strongest RUL correlations include s11 at -0.696, s4 at -0.679, s12 at $+0.672$ and s7 at $+0.657$. A negative correlation with RUL indicates that the reading tends to rise as failure gets closer. Sensors s9 and s14 have weaker individual correlations, -0.390 and -0.307. Because they differ from the main correlated family, weak marginal association alone is insufficient grounds for removal.

The modelling handoff retains all 14 candidate sensors, excludes identifiers and targets, keeps degradation outliers, standardises the appropriate model families and groups validation by engine. These decisions define the reference pipeline before the final test is scored. Supervised exploration across all training engines remains an adaptive choice, which is relevant when interpreting cross-validation estimates.

IX. Modelling protocol and baseline

9.1 Grouped validation and prediction scores

A shared helper clones the estimator for each fold, fits it on the training engines and returns both fold metrics and an out-of-fold score for every training row. It asserts that training and validation engine sets are disjoint and that every row receives a finite score. Fold scores and pooled out-of-fold scores answer different aggregation questions : the former average over engine-group folds, whereas the latter evaluate the assembled row-level predictions.

Probability-capable models return the class-one probability and use a default threshold of 0.5. The RBF SVM uses a decision-function score and a default threshold of zero. ROC AUC and average precision assess ranking across thresholds ; recall, precision and F1 depend on the chosen alarm threshold. SVM margins should not be interpreted as calibrated probabilities.

Metric	**Definition**	**Interpretation**
Accuracy	(TP + TN) / all observations	Can be high when the model ignores the positive class
Recall	TP / (TP + FN)	Fraction of imminent failures detected
Precision	TP / (TP + FP)	Fraction of alarms that correspond to imminent failure
F1	2PR / (P + R)	Harmonic balance between precision and recall
ROC AUC	Area under the ROC curve	Ranking of positive versus negative observations
Average precision	Recall-weighted average of precision	Summarises the precision-recall curve ; depends on prevalence

Table 9 – Reported metrics and their interpretation

9.2 Always healthy reference

The baseline is a DummyClassifier using the training class prior. At the default threshold it always predicts healthy. In grouped training validation, it obtains about 85.0 percent accuracy, zero recall, ROC AUC 0.50 and average precision 0.150. These results demonstrate why accuracy cannot be the principal success measure for this problem. The high accuracy comes from the majority class rather than useful failure detection.

No resampling or synthetic examples are added. Class weighting is evaluated separately. The standardised pipelines keep preprocessing inside the fold. All operating-point and model choices are made from training experiments ; the test outcome is examined only after the chosen estimator and threshold have been set for this study.

X. Comparison of the classifier families

Model	Initial settings and preprocessing
Logistic regression	StandardScaler; balanced weights; C = 1; max_iter = 1000
Gaussian naive Bayes	StandardScaler; default GaussianNB smoothing
k nearest neighbours	StandardScaler; 25 neighbours; uniform voting
RBF support vector machine	StandardScaler; RBF kernel; balanced weights
Decision tree	Maximum depth 6; balanced weights; random_state = 0
Random forest	200 trees; minimum leaf size 5; balanced weights; random_state = 0
Histogram gradient boosting	Balanced weights; default depth and learning settings; random_state = 0

TABLE 10 – Initial classifier configurations

The comparison includes a regularised linear model, a conditional Gaussian model, a distance-based method, a kernel method, a single tree and two tree ensembles. This range tests whether the observed nonlinear degradation patterns require a more flexible classifier. Balanced weights are used where supported; naive Bayes and k nearest neighbours do not have the same class-weight option.

Model	Recall	Precision	F1	ROC AUC	AP
Naive Bayes	0.965 (0.022)	0.702 (0.021)	0.813 (0.013)	0.990 (0.002)	0.952 (0.010)
Logistic regression	0.946 (0.025)	0.728 (0.024)	0.822 (0.017)	0.989 (0.002)	0.949 (0.008)
Random forest	0.905 (0.031)	0.810 (0.025)	0.854 (0.011)	0.989 (0.002)	0.947 (0.010)
Gradient boosting	0.923 (0.028)	0.772 (0.033)	0.840 (0.014)	0.989 (0.002)	0.947 (0.009)
k-NN	0.833 (0.045)	0.899 (0.014)	0.864 (0.021)	0.985 (0.005)	0.940 (0.015)
Decision tree	0.937 (0.014)	0.665 (0.019)	0.777 (0.011)	0.967 (0.008)	0.875 (0.015)
SVM (RBF)	0.961 (0.023)	0.707 (0.028)	0.814 (0.016)	0.980 (0.003)	0.863 (0.019)

TABLE 11 – Initial grouped five-fold cross-validation scores with fold standard deviations

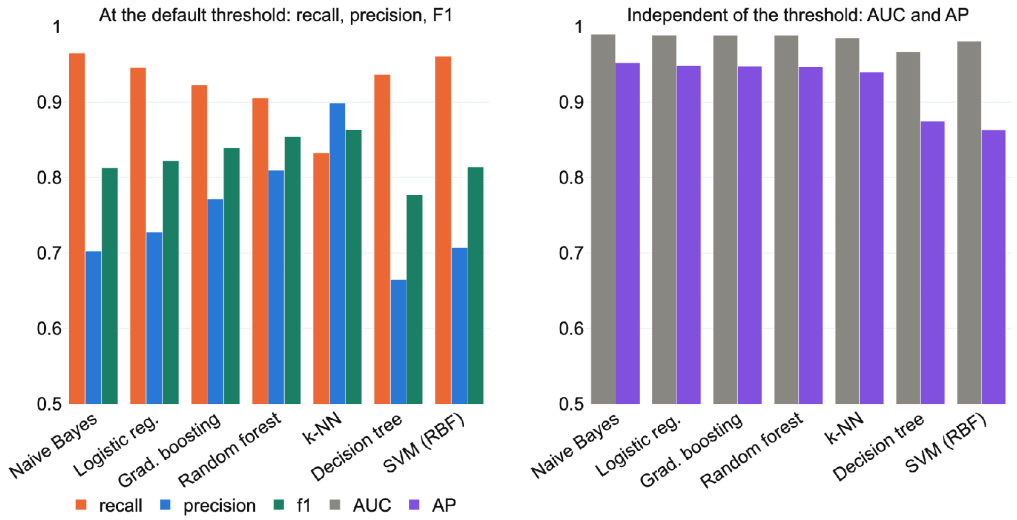


FIGURE 23 – Initial model comparison showing default-threshold measures and ranking measures separately with a zoomed vertical range of 0.5 to 1.0
Source : notebook section 15.2.

A default threshold is the cutoff applied to a model score to turn it into a class prediction. For example, logistic regression predicts imminent failure when its class-one probability reaches 0.5, whereas the RBF SVM uses zero on its decision-function score. Recall, precision and F1 depend on this cutoff ; ROC AUC and average precision assess ranking across thresholds.

Six initial models reach ROC AUC values around 0.98 to 0.99. The decision tree is weaker. Logistic regression already performs near the stronger families, with AP 0.949 and ROC AUC 0.989. k nearest neighbours favours precision at the default threshold, whereas weighted models generally favour recall. These are partly differences of operating point, not proof that one model has universally better ranking.

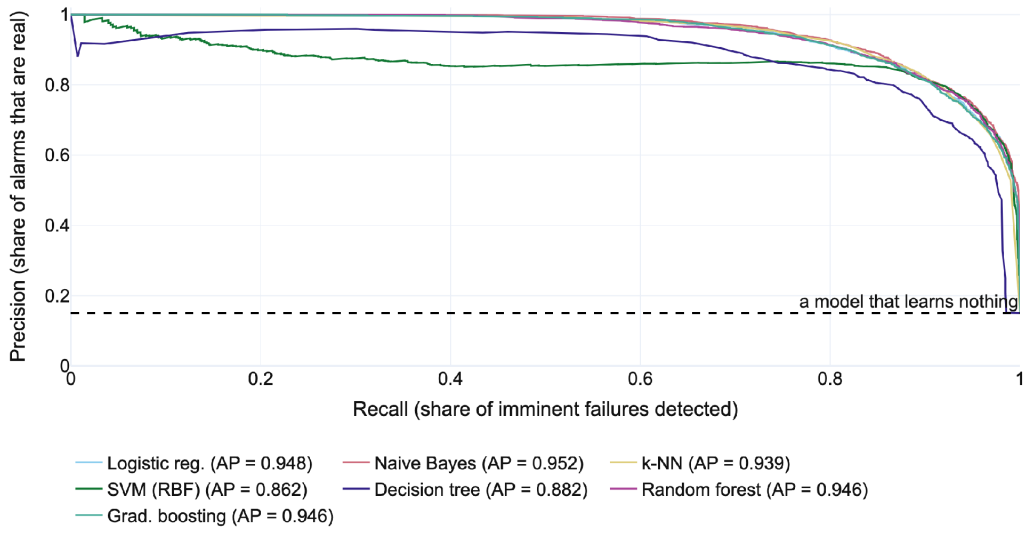


FIGURE 24 – Out-of-fold precision-recall curves for the initial classifiers
Source : notebook section 15.3.

The precision-recall curves show the trade-off available when each model threshold changes. The horizontal reference is the positive-class prevalence. Threshold comparisons should use continuous scores rather than default hard labels alone. These curves are pooled over training out-of-fold rows ; they are not endpoint test curves or calibrated maintenance-risk curves.

XI. Sensor ablation and class weighting

11.1 Testing whether s9 and s14 can be removed

We compare the full sensor set with a reduced 12-sensor set that removes s9 and s14. Five representative model families are evaluated using three engine shuffles, seeds zero, one and two, giving 15 paired folds per model and 150 fits overall. The same model and split are used for each pair so the AP difference isolates the feature-set change as closely as this protocol permits.

Model	AP with 14 sensors	AP change with 12 sensors	Folds where 12 is better
Logistic regression	0.9504	-0.0177	0 / 15
Naive Bayes	0.9543	-0.0296	0 / 15
k-NN	0.9414	-0.0207	0 / 15
Random forest	0.9477	-0.0197	0 / 15
Gradient boosting	0.9481	-0.0209	0 / 15

Table 12 – Paired effect of removing s9 and s14

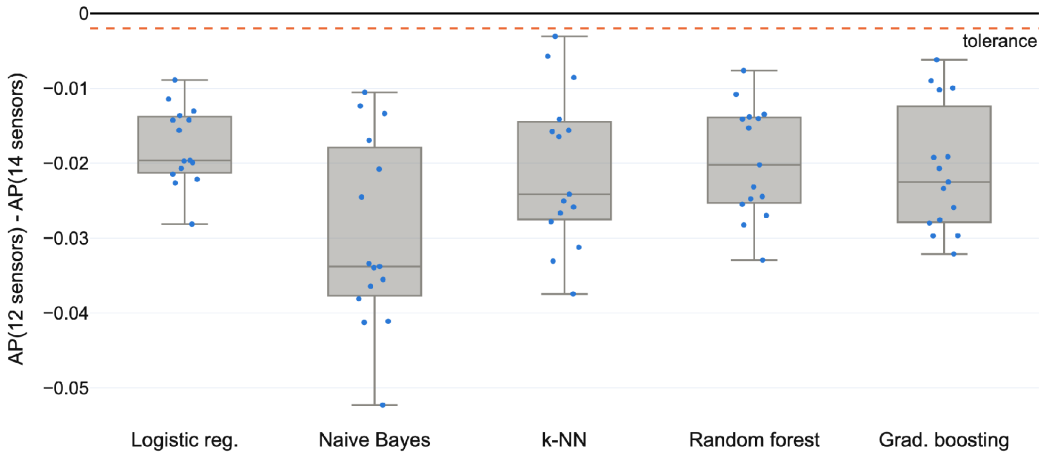


Figure 25 – Paired AP change when s9 and s14 are removed with the predeclared tolerance
Source : notebook section 16.2.

The rule allows removal only when at least four of the five models have a mean AP reduction smaller than 0.002. None meets that condition. Every model has lower AP on all 15 folds after removal ; mean losses range from 0.0177 to 0.0296. We retain the 14-sensor set. These repeated folds are correlated because they reuse engines, so the plot is evidence of consistency rather than 15 independent replications or a formal significance test.

11.2 Effect of balanced class weights

Balanced class weights compensate for the approximate 5.7-to-one negative-to-positive ratio in training. Four supported model families are compared with and without weighting. Logistic regression recall increases from 0.841 to 0.946, while precision decreases from 0.882 to 0.728. Its ROC AUC and AP remain approximately unchanged. The decision tree also changes ranking quality, so weights do not merely translate a threshold for every estimator.

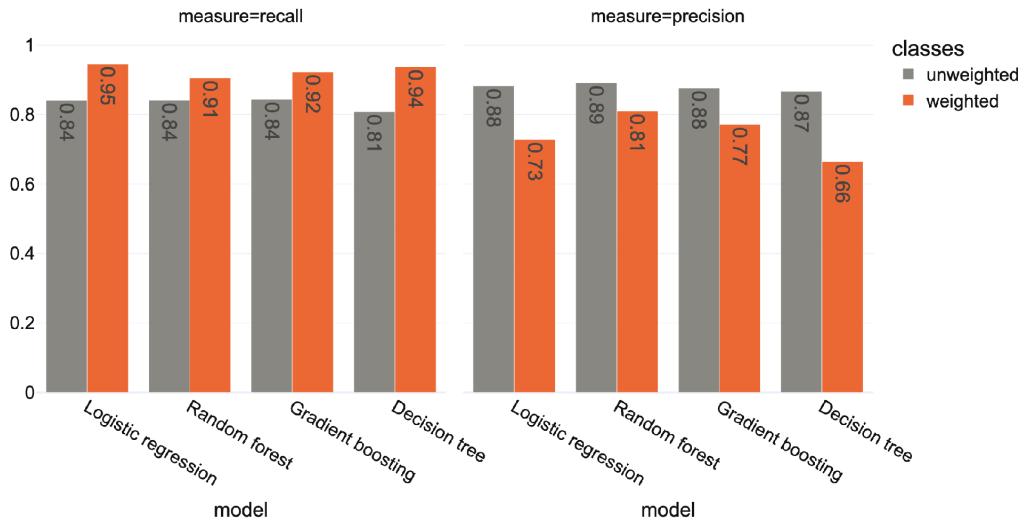


FIGURE 26 – Effect of class weighting on recall and precision at the default threshold
Source : notebook section 17.1.

Model	Weights	Recall	Precision	F1	ROC AUC	AP
Logistic regression	unweighted	0.841	0.882	0.861	0.989	0.949
Logistic regression	weighted	0.946	0.728	0.822	0.989	0.949
Random forest	unweighted	0.842	0.891	0.865	0.989	0.950
Random forest	weighted	0.905	0.810	0.854	0.989	0.947
Gradient boosting	unweighted	0.844	0.877	0.859	0.989	0.947
Gradient boosting	weighted	0.923	0.772	0.840	0.989	0.947
Decision tree	unweighted	0.808	0.867	0.836	0.972	0.897
Decision tree	weighted	0.937	0.665	0.777	0.967	0.875

TABLE 13 – Weighted and unweighted grouped validation results

We keep balanced weights as the reference configuration and subsequently choose the alarm threshold explicitly. The trade-off remains a modelling preference : a weighted fit does not by itself specify how many false alarms a maintenance service can tolerate. An application-specific cost function and probability calibration would be needed for a financial or safety decision.

XII. Hyperparameter tuning

GridSearchCV uses average precision as the search score and five engine-grouped folds with seed zero. The seven grids contain 48 candidate configurations in total, giving 240 candidate-fold fits. refit=False avoids an extra full-data refit at this stage ; the best settings are applied to cloned

estimators, which are refitted later. The finite grids define the scope of the comparison rather than proving that each family has reached its global optimum.

Model	Grid	Trials
Logistic regression	C : 0.01, 0.1, 1, 10, 100	5
Naive Bayes	var_smoothing : 1e-9, 1e-6, 1e-3, 0.1	4
k nearest neighbours	Neighbours : 5, 15, 25, 50, 100 ; weights : uniform or distance	10
RBF SVM	C : 0.1, 1, 10 ; gamma : 0.01, 0.03, 0.1	9
Decision tree	max_depth : 3, 5, 8 ; min_samples_leaf : 20 or 100	6
Random forest	min_samples_leaf : 5, 20, 50 ; max_features : sqrt or 0.5	6
Gradient boosting	learning_rate : 0.03 or 0.1 ; max_leaf_nodes : 8 or 31 ; l2_regularization : 0 or 10	8

Table 14 – Hyperparameter search ranges

Model	Selected settings	Search AP
Logistic regression	C = 100	0.9486
Naive Bayes	var_smoothing = 0.1	0.9525
k-NN	n_neighbors = 100 ; weights = distance	0.9517
SVM (RBF)	C = 1 ; gamma = 0.01	0.9532
Decision tree	max_depth = 8 ; min_samples_leaf = 100	0.9035
Random forest	max_features = sqrt ; min_samples_leaf = 5	0.9467
Gradient boosting	l2_regularization = 10.0 ; learning_rate = 0.1 ; max_leaf_nodes = 8	0.9506

Table 15 – Selected grid settings and search AP

12.1 Reevaluation on changed engine shuffles

The default and tuned estimators are scored again using seeds one and two, resulting in ten folds per model version. This changes how the same 100 training engines are partitioned. It is a stability check and a comparison under new splits, not an independent holdout evaluation : the hyperparameter search has already used those engines. Non-nested model selection can introduce optimism into the reported validation performance [3].

Model	Recall	Precision	F1	ROC AUC	AP	AP SD	AP gain
Naive Bayes	0.961	0.712	0.815	0.990	0.955	0.009	0.000
SVM (RBF)	0.955	0.722	0.820	0.990	0.955	0.010	0.094
k-NN	0.829	0.904	0.863	0.989	0.953	0.010	0.011
Logistic regression	0.945	0.730	0.822	0.990	0.951	0.011	0.000
Gradient boosting	0.941	0.741	0.827	0.989	0.951	0.009	0.003
Random forest	0.900	0.812	0.852	0.989	0.948	0.010	0.000
Decision tree	0.939	0.690	0.794	0.980	0.908	0.012	0.025

TABLE 16 – Tuned classifier comparison on ten reshuffled grouped folds

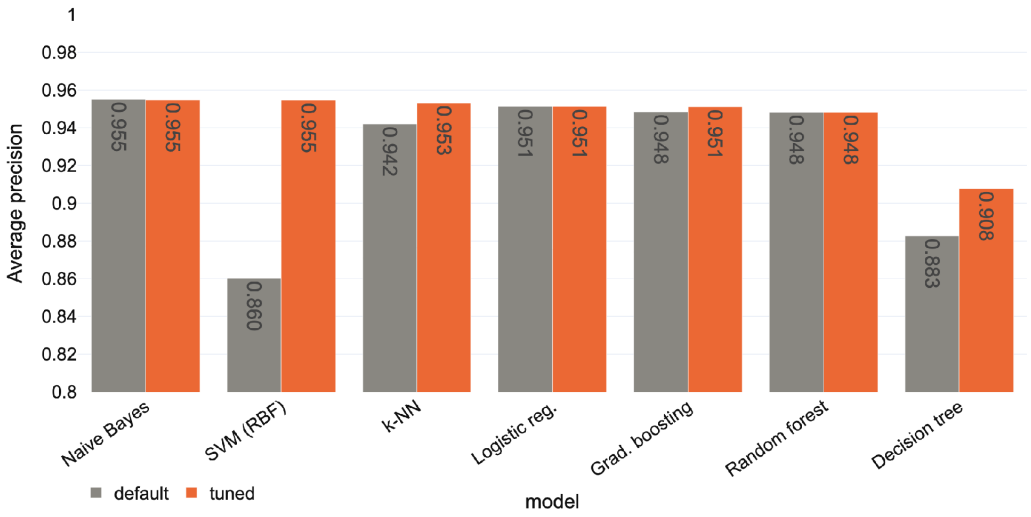


FIGURE 27 – Average precision of default and tuned estimators on two new engine-group shuffles with a zoomed vertical range of 0.8 to 1.0

Source : notebook section 18.2.

Tuning matters most for the SVM, with an AP gain of about 0.094. The decision tree gains 0.025 and k nearest neighbours gains 0.011. Most other changes are very small. The leading tuned models have AP near 0.951 to 0.955, while fold standard deviations are around 0.009 to 0.011. These standard deviations describe variation across dependent folds and are not confidence intervals for a model-to-model performance difference.

XIII. Model selection and alarm threshold

13.1 Choosing the reference estimator

We first find the highest reevaluated AP and then consider models within 0.005 of that value. Among those candidates, the predeclared simplicity order chooses logistic regression before more complex families. Logistic regression, naive Bayes, k nearest neighbours, RBF SVM and gradient boosting meet the tolerance. Logistic regression is selected with C = 100, balanced class weights and a StandardScaler pipeline.

The 0.005 tolerance is a practical selection heuristic, not a statistical equivalence test. The decision favours a transparent reference model with low fitting cost while retaining competitive

ranking performance. It does not establish that degradation is physically linear or that other model families cannot improve with additional features or a different search.

13.2 Selecting an operating point from training predictions

$$F_2 = \frac{5PR}{4P + R}$$

P denotes precision and R denotes recall. We evaluate F2 over the precision-recall thresholds from the selected model out-of-fold scores using seed one. Its beta parameter emphasises recall more strongly than precision, but it does not encode a specific monetary cost ratio. The maximising threshold is approximately 0.592. The displayed value is rounded ; comparisons in the notebook use the full floating-point threshold.

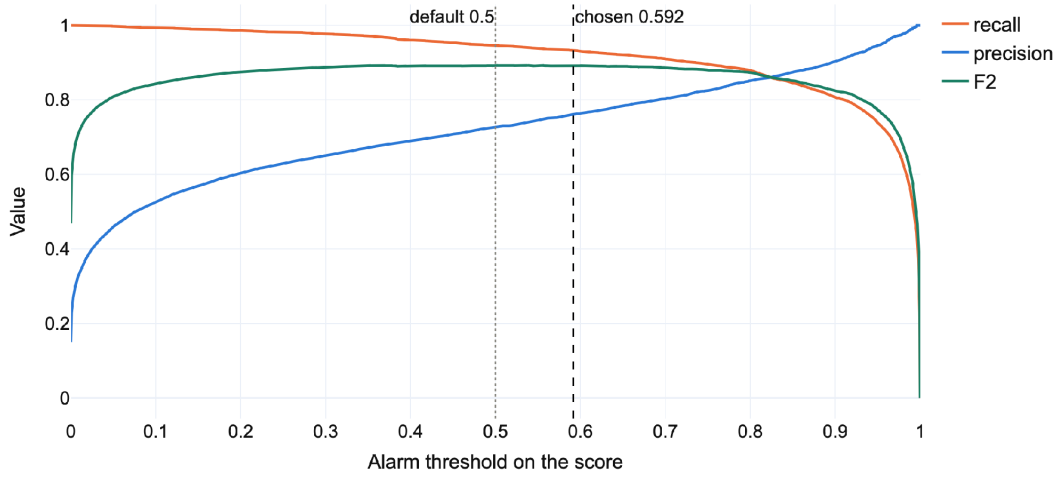


FIGURE 28 – Training out-of-fold recall precision and F2 across the logistic regression alarm threshold
Source : notebook section 19.2.

Threshold	Recall	Precision	F1	F2
default (0.5)	0.946	0.726	0.822	0.892
chosen (0.592)	0.934	0.762	0.839	0.893

TABLE 17 – Effect of threshold selection on pooled training out-of-fold predictions

Moving from 0.5 to the chosen threshold changes recall from 0.946 to 0.934 and precision from 0.726 to 0.762. F2 changes only from 0.892 to 0.893, indicating a broad optimum rather than a uniquely reliable point. The threshold is fixed before the test results are consulted. Its F2 on the same scores used to select it is descriptive and can be optimistic.

XIV. Final evaluation and uncertainty

14.1 Refitting and testing on unseen engines

Each tuned estimator is refitted on all 20,631 training rows. The primary test evaluates the last recorded row of each of the 100 test engines, of which 25 satisfy the 30-cycle label. The comparison displays the tuned families at their respective default thresholds and the chosen logistic model at its selected threshold. The table is reported for transparency ; it is not used to change the selected family.

Model and threshold	Recall	Precision	F1	ROC AUC	AP
Naive Bayes	0.920	0.852	0.885	0.987	0.962
k-NN	0.680	0.895	0.773	0.985	0.957
Random forest	0.800	0.909	0.851	0.984	0.956
SVM (RBF)	0.880	0.846	0.863	0.982	0.952
Gradient boosting	0.840	0.840	0.840	0.979	0.945
Logistic regression	0.880	0.815	0.846	0.978	0.944
Logistic regression selected 0.592	0.840	0.875	0.857	0.978	0.944
Decision tree	0.880	0.815	0.846	0.970	0.908
Always healthy baseline	0.000	0.000	0.000	0.500	0.250

TABLE 18 – Final endpoint test results

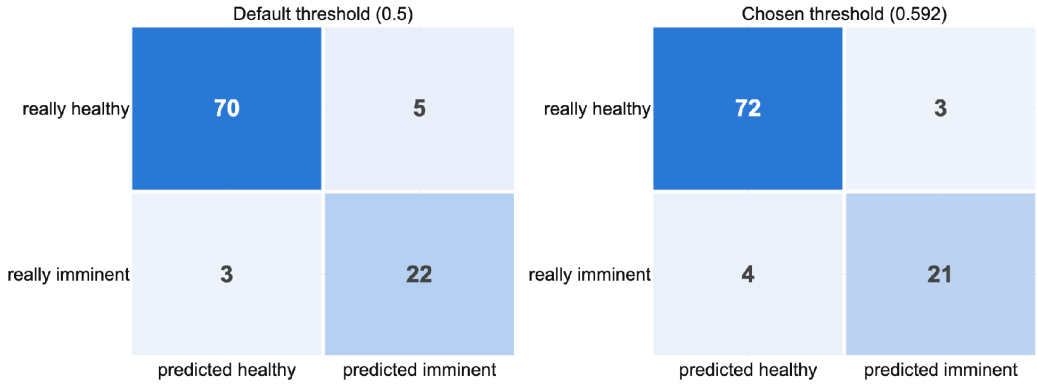


FIGURE 29 – Confusion matrices for the selected model at thresholds 0.5 and approximately 0.592
Source : notebook section 20.2.

At the chosen threshold, the confusion matrix contains 72 true negatives, three false positives, four false negatives and 21 true positives. Recall is $21/25 = 0.840$ and precision is $21/24 = 0.875$. F1 is 0.857 and accuracy is 0.930. Raising the threshold from 0.5 avoids two false alarms but misses one additional positive engine. One detected positive changes recall by four percentage points in this test.

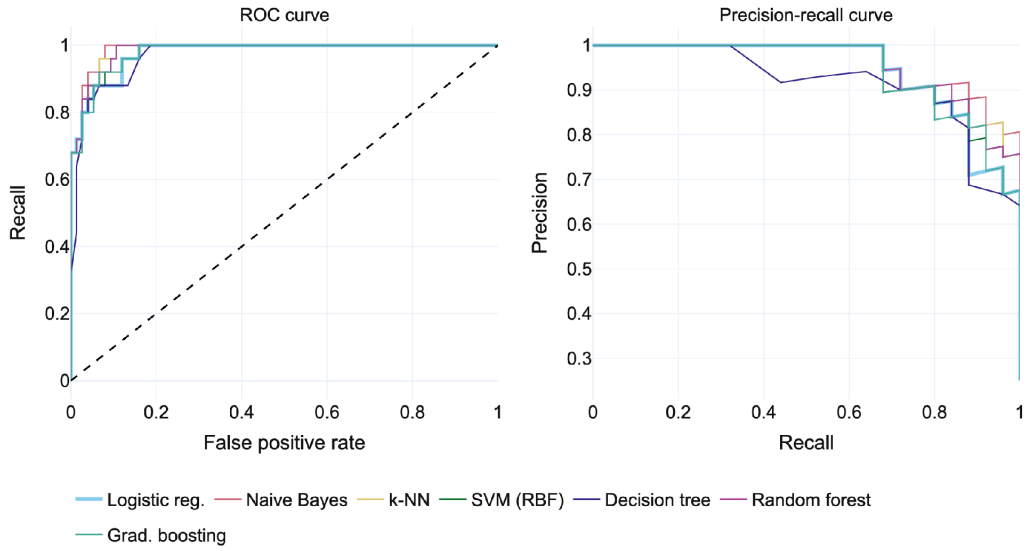


FIGURE 30 – Endpoint ROC and precision-recall curves for all tuned classifiers with the chosen model emphasised

Source : notebook section 20.2.

The chosen model ROC AUC is 0.978 and its AP is 0.944. Naive Bayes has the highest endpoint AP, 0.962, but choosing it after examining the test table would turn that test into another selection set. Differences on this small sample should not be overinterpreted. The observed performance of the previously fixed logistic model is the primary result.

14.2 Bootstrap uncertainty

We draw the 100 endpoint engines with replacement 2,000 times using random seed zero and recompute the fixed-model metrics. Samples with only one class are skipped for AUC. The 2.5th and 97.5th percentiles form the reported 95 percent bootstrap intervals. Because the model and threshold are held fixed, these intervals describe sampling uncertainty among test engines, not the full uncertainty of training, selecting and retuning the pipeline.

Metric	Test estimate	95 percent lower	95 percent upper
recall	0.840	0.667	0.963
precision	0.875	0.727	1.000
f1	0.857	0.727	0.945
auc	0.978	0.949	0.995
ap	0.944	0.871	0.988

TABLE 19 – Endpoint metrics and percentile bootstrap intervals

Recall ranges from 0.667 to 0.963 and precision from 0.727 to 1.000. The wider intervals reflect the small number of positives. The row-level validation and endpoint test estimates lie within broadly compatible ranges, but the populations and aggregation differ, so this is not proof that the validation procedure is unbiased.

14.3 All rows compared with endpoints

Population	Recall	Precision	F1	ROC AUC	AP
all 13,096 test rows	0.804	0.624	0.703	0.989	0.789
last row of each engine (100)	0.840	0.875	0.857	0.978	0.944
cross-validation on train	0.934	0.762	0.839	0.989	0.947

TABLE 20 – Three evaluation populations for the selected model

On all 13,096 test rows the chosen model has recall 0.804, precision 0.624 and F1 0.703. Its ranking remains strong, with ROC AUC 0.989, but AP falls to 0.789 and precision is lower than at the endpoints. Only 2.535 percent of these rows are positive. False alarms among many negative rows can therefore outnumber detections even when discrimination is strong. Monitoring rows from the same engine are dependent ; no row-independent confidence interval is claimed for this secondary view.

XV. Error analysis and continuous monitoring

15.1 The seven endpoint errors

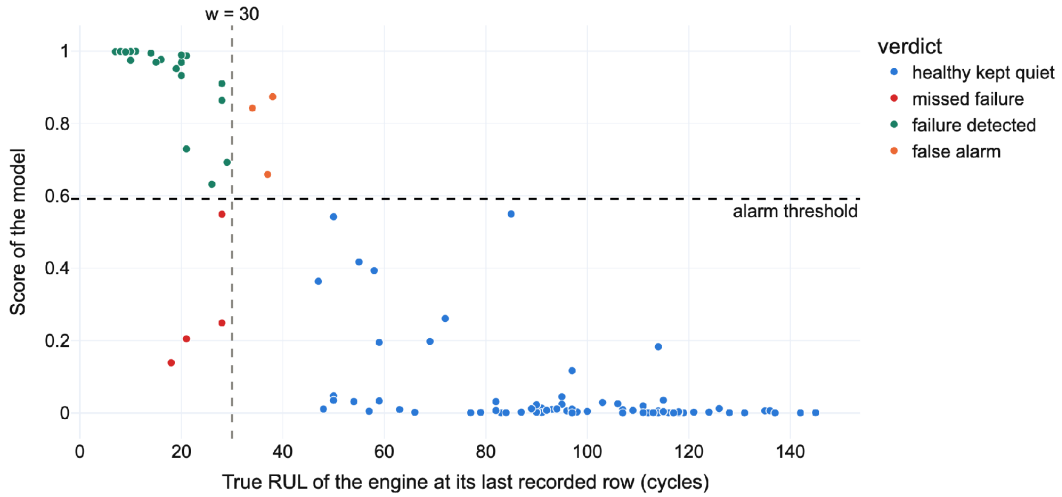


FIGURE 31 – Selected logistic regression score against true remaining life for each test engine
Source : notebook section 21.1.

Engine	Cycle	True RUL	Score	Error	Distance from 30
18	133	28	0.249	missed failure	2
64	168	28	0.549	missed failure	2
77	162	34	0.842	false alarm	4
58	176	37	0.659	false alarm	7
91	234	38	0.874	false alarm	8
37	121	21	0.205	missed failure	9
41	123	18	0.139	missed failure	12

TABLE 21 – Incorrect endpoint classifications and distance from the label boundary

The missed failures occur at RUL 28, 28, 21 and 18. The false alarms occur at RUL 34, 37 and 38. All seven errors are within 12 cycles of the binary boundary. This is consistent with a gradual degradation process observed through noisy snapshots. It does not make the errors harmless : the model still misses four imminent failures, and a true operational policy would need to assess their consequences.

15.2 Alarm frequency along the test histories

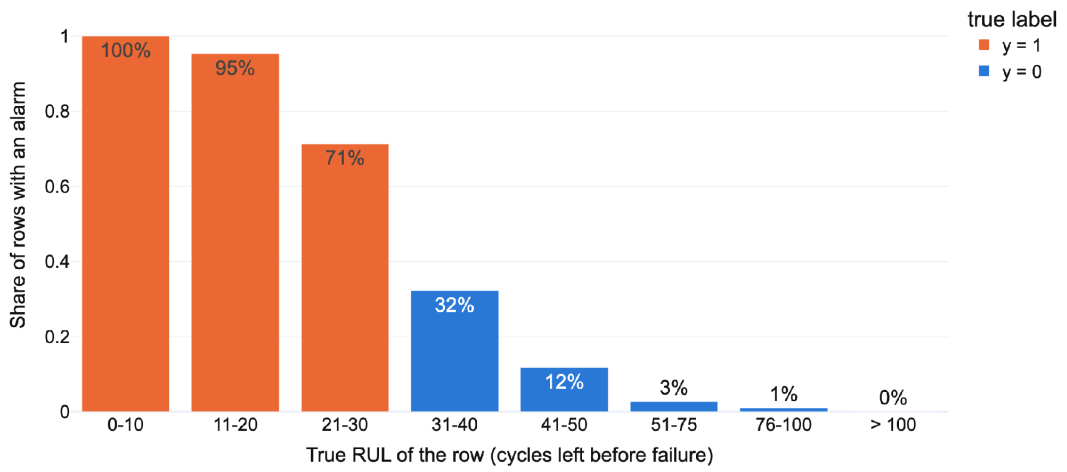


FIGURE 32 – Alarm proportion by true remaining-life band across every test row
Source : notebook section 21.2.

RUL band in cycles	Rows with alarm percent	Rows
0-10	100.0%	17
11-20	95.3%	106
21-30	71.3%	209
31-40	32.2%	264
41-50	11.7%	290
51-75	2.7%	973
76-100	1.0%	1335
> 100	0.0%	9902

TABLE 22 – Alarm proportions and sample sizes in the RUL bands

The alarm is present on all 17 rows with RUL zero to ten and on 95.3 percent of the 106 rows with RUL 11 to 20. It is present on 71.3 percent of rows with RUL 21 to 30, then falls to 32.2 percent in the band 31 to 40 and 11.7 percent in 41 to 50. This transition is smoother than the label boundary and explains much of the continuous-monitoring false-alarm burden.

Roughly 85 false alarms occur in the 31 to 40 band and 34 in 41 to 50, based on the displayed rounded rates and band sizes. These are alerts shortly before the defined positive window, but they remain false positives under the stated task. An operational study should evaluate alert persistence, lead time and alarms per engine, rather than count every positive cycle as an independent maintenance event.

XVI. Conclusions limitations

The reference workflow yields a useful condition-based FD001 classifier with clear separation from the always-healthy baseline. Data quality checks pass, degradation patterns support the candidate sensors, grouped validation avoids splitting the same engine across training and validation, and the final logistic model detects 21 of 25 positive test engines with three false alarms. Its simplicity is a selection choice supported by competitive validation ranking, not a claim of universal superiority.

16.1 Limitations of the current study

Simulation and scope. FD001 has one condition and one fault mode. The report does not establish performance on FD002 to FD004, other engine designs or real aircraft fleets.

Single-row inputs. The classifier does not use previous sensor history. Adjacent-cycle trends, robust moving averages and individual-engine baselines could reduce ambiguity, but have not yet been validated here.

Adaptive validation. Feature investigation, hyperparameter selection, family selection and threshold optimisation reuse the training engines. Changing fold seeds does not make the reevaluation independent. A nested grouped protocol would give a stronger estimate of the full model-selection procedure.

Small endpoint test. Only 100 engines and 25 positives are available. Bootstrap intervals are wide, and apparent model differences should not be treated as stable rankings.

Threshold and prevalence. The 30-cycle warning window and the F2 objective are chosen conventions. The threshold is not a calibrated maintenance cost optimum, and precision changes when prevalence changes.

Repeated test use. This test set has now been inspected for performance, errors and monitoring behaviour. Further iterations should disclose that reuse and reserve a new holdout if they require an independent final assessment.